Bi. – Mô hình hoá và không gian trạng thái

→ Tọa độ các thực thể

- Pacman : $P = (1,1)$
- Đích : $G = (1,7)$
- Thức ăn ban đầu ($m = 2$ viên) $f_a = (3,3)$, $f_b = (3,5)$

→ Số ô đi được ($|V|$)

- Hàng 1: 6 ô gồm (1,1) , (1,2) , (1,3) , (1,5) , (1,6) , (1,7)
- Hàng 2: 6 ô gồm (2,1) , (2,2) , (2,3) , (2,5) , (2,6) , (2,7)
- Hàng 3: 7 ô gồm (3,1) ; (3,2) ; (3,3); (3,4) ; (3,5); (3,6); (3,7).

Tổng số ô hợp lệ $|V| = 6 + 6 + 7 = 19$ ô

→ Cận trên số trạng thái

- Mỗi trạng thái $s = (p, R)$ với $p \in V$ và $R \subseteq \{F_a, F_b\}$
- Số trạng thái tối đa: $|S| \leq |V| \times 2^m = 19 \times 2^2 = 76$ (tt)

– Tính giá trị Heuristic (h_F, h_V)

→ Khoảng cách manhattan: $d(x,y) = |x_x - x_y| + |c_x - c_y|$

→ Khoảng cách từ các viên thức ăn về đích $G(1,7)$:

$d(F_a, G) = 6$, $d(F_b, G) = 4$

Scanned with CamScanner

$S = (p, R)$

$f \in R$ ~~$\cup$~~ $\{d$

Trạng thái $S = (p, R)$	$d(p, F_a)$	$d(p, F_b)$	$d(p, G)$	$h_F(p, R) = \max_{f \in R} d(p, f)$	$h_V(p, R) =$ $\max[\{d(p, G)\} \cup \{d(p, f) + d(f, G)\}]$
Gốc $S_0 =$ $((1,1), \{F_a, F_b\})$	4	6	6	$\max(4, 6) = 6$	$\max(6, 4+6, 6+4) = 10$
Con 1(R): $S_1 =$ $((1,2), \{F_a, F_b\})$	3	5	5	$\max(3, 5) = 5$	$\max(5, 3+6, 5+4) = 9$
Con 2(0): $S_2 =$ $((2,1), \{F_a, F_b\})$	3	5	7	$\max(3, 5) = 5$	$\max(7, 3+6, 5+4) = 9.$

Scanned with CamScanner

— Đường đi tìm được:

↦ Chuỗi hành động ~~RR DD RRRRDD~~ RRDDRRRRUU.

↦ Diễn biến trạng thái:

- Bước 0: (1,1), R = {Fa, Fb} mask 3
- Bước 1: (1,2), R = {Fa, Fb}
- Bước 2: (1,3), R = {Fa, Fb}
- Bước 3: (2,3), R = {Fa, Fb}
- Bước 4: (3,3), → ăn Fa còn lại R = {Fb} (mask 2)
- Bước 5: (3,4), R = {Fb}
- Bước 6: (3,5), → ăn Fb, còn lại R = ∅ (mask 0)
- Bước 7: (3,6), R = ∅
- Bước 8: (3,7), R = ∅
- Bước 9 (2,7), R = ∅
- Bước 10 (1,7) = G, R = ∅ → thoả mãn điều kiện dích

⇒ Tổng chi phí g = 10 bước (chi phí mỗi bước c = 1)

— Chứng minh tính tối ưu bằng cận dưới:

↦ Mọi hành trình hợp lệ đều xuất phát từ P, qua 2 điểm Fa, Fb và kết thúc tại G. Theo bất đẳng thức tam giác của khoảng cách Manhattan

- Nếu ăn theo thứ tự Fa → Fb:

  Chi phí ≥ d(P, Fa) + d(Fa, Fb) + d(Fb, G) = 4+2+4 = 10
- Nếu ăn theo thứ tự Fb → Fa:

  Chi phí ≥ d(P, Fb) + d(Fb, Fa) + d(Fa, G) = 6+2+6 = 14

↦ Cận dưới lý thuyết không thể vượt qua:

$$Cost^* \geq \min(10, 14) = 10$$

⇒ Đường đi thực thi RR DD RRRRUU hoàn toàn hợp lệ và đạt chi phí đúng bằng 10 → đường đi này tối ưu toàn cục.

Scanned with CamScanner

B 2

1.

- Bảng 1: UCS ($h = 0$, $f = g$)

Lần lấy	Trạng thái lấy ra (c; R) (n, c, mask)	g	h	f	Cha	Con mới sinh ra (theo thứ tự RDLU)	OPEN sau xử lý (theo thứ tự sẽ lấy
1	(3; ab) (1, 3, 3) (tickret = 0)	0	0	0	None	R: (4; a) / (1, 4, 1) [g = 1, h = 0 f = 1, tickret = 1) (cần b, xoá bớt 1) L: (2; ab) / (1, 2, 3) [g = 1, h = 0, f = 1, tickret = 2]	1. (4; a) / (1, 4, 1) [g = 1, h = 0, f = 1 th = 1] 2. (2; ab) / (1, 2, 3) [g = 1, h = 0 f = 1, th = 2]
2	(4; a) (1, 4, 1) (th = 1)	1	0	1	(3; ab) (đi R)	R: (5; a) / (1, 5, 1) [g = 2, h = 0 f = 2, th = 3] L (3; a) / (1, 3, 1) [g = 2, h = 0 f = 2, th = 4) quay lại cột 3 với mask mới)	1. (2; ab) / (1, 2, 3) [g = 1, h = 0 f = 1, th = 2] 2. (5; a) / (1, 5, 1) [g = 2, h = 0 f = 2, th = 3] 3. (3; a) / (1, 3, 1) [g = 2, h = 0, f = 2, th = 4]
3	(2; ab) (1, 2, 3) (th = 2)	1	0	1	(3; ab) (đi L)	(R về (3; ab) có $g_{ms} = 2$ & best g = 0 nên loại) L (1; b) / (1, 1, 2) [g = 2 h = 0, f = 2, th = 5] (cần a, xoá bớt 0)	1. (5; a) / (1, 5, 1) [g = 2, h = 0 f = 2, th = 3] 2 (3; a) / (1, 3, 1) [g = 2, h = 0 f = 2, th = 4] 3. (1; b) / (1, 1, 2) [g = 2, h = 0, f = 2, th = 5]
4	(5; a) (1, 5, 1) (th = 3)	2	0	2	(4; a) (đi R)	R (6; a) / (1, 6, 1) [g = 3, h = 0 f = 3, th = 6) (qua G lại còn a) (L về (4; a) có $g_{ms} = 3$ & best g = 1 nên loại)	1. (3; a) / (1, 3, 1) [g = 2, h = 0 f = 2, th = 4] 2. (1; b) / (1, 1, 2) [g = 2, h = 0 f = 2, th = 5] 3. (6; a) / (1, 6, 1) [g = 3, h = 0, f = 3, th = 6]
5	(3; a) (1, 3, 1) (th = 4)	2	0	2	(4; a) (đi L)	(R về (4; a) có $g_{ms} = 3$ & best g = 1 nên loại) L (2; a) / (1, 2, 1) [g = 3 h = 0, f = 3, th = 7]	1. (1; b) / (1, 1, 2) [g = 2 h = 0, f = 2, th = 5] 2. (6; a) / (1, 6, 1) [g = 3 h = 0, f = 3, th = 6] 3. (2; a) / (1, 2, 1) [g = 3 h = 0, f = 3, th = 7]
6	(1; b) (1, 1, 2) th = 5	2	0	2	(2; ab) (đi L)	R (2, b) / (1, 2, 2) [g = 3, h = 0, f = 3, th = 8) (quay lại cột 2 với mask mới) - Hướng L chạm tường cột 0	1. (6; a) / (1, 6, 1) [g = 3, h = 0, f = 3, th = 6] 2. (2; a) / (1, 2, 1) [g = 3, h = 0, f = 3, th = 7] 3. (2, b) / (1, 2, 2) [g = 3, h = 0, f = 3, th = 8]

- Bảng 2: A_V ($h = h_v$, $f = g + h_v$)

Lần lấy	Trạng thái lấy ra (c; R) (n, c, mask)	g_v	h	f	Cha	Con mới sinh ra (theo thứ tự RDLU)	OPEN sau xử lý
1	(3; ab) (1, 3, 3) (th = 0)	0	7	7	None	R (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1) (cần b, xoá bớt 1) L (2; ab) / (1, 2, 3) [g = 1, h = 6, f = 7, th = 2)	1. (2; ab) / (1, 2, 3) [g = 1, h = 6, f = 7, th = 2] 2. (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1]

2	(2; ab) (1,2,3) (th = 2)	1	6	7	(3; ab) (đi L)	- (Hướng R về (3; ab] có g_{mn} = 2 & best g = 0 nên loại) - L: (1; b) / (1, 1, 2) [g = 2, h = 5, f = 7, th = 3] (ăn a, xóa b về 0)	1. (1; b) / (1, 1, 2) [g = 2, h = 5, f = 7, th = 3] 2. (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1]
3	(1; b) (1, 1, 2) (th = 3)	2	5	7	(2; ab) (đi L)	- R (2; b) / (1, 2, 2) [g = 3, h = 4, f = 7, th = 4] (quay lại cột 2 với mark 2) - (Hướng L chạm tường cột 0)	1. (2; b) / (1, 2, 2) [g = 3, h = 4, f = 7, th = 4] 2. (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1]
4	(2; b) (1, 2, 2) (th = 4)	3	4	7	(1; b) (đi R)	- R (3; b) / (1, 3, 2) [g = 4, h = 3, f = 7, th = 5] - (Hướng L về (1; b) có g_{mn} = 4 & best g = 2 bị loại)	1. (3; b) / (1, 3, 2) [g = 4, h = 3, f = 7, th = 5] 2. (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1]
5	(3; b) (1, 3, 2) (th = 5)	4	3	7	(2; b) (đi R)	- R (4, -) / (1, 4, 0) [g = 5, h = 2, f = 7, th = 6] (ăn b, mark về 0) - (Hướng L về (2; b] có g_{mn} = 5 & best g = 3 bị loại	1. (4; -) / (1, 4, 0) [g = 5, h = 2, f = 7, th = 6] 2. (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1]
6	(4; -) (1, 4, 0) (th = 6)	5	2	7	(3; b) (đi R)	R (5; -) / (1, 5, 0) [g = 6, h = 1, f = 7, th = 7] - L (3; -) / (1, 3, 0) [g = 6, h = 3, f = 9, th = 8]	1. (5; -) / (1, 5, 0) [g = 6, h = 1, f = 7, th = 7] 2. (4; a) / (1, 4, 1) [g = 1, h = 8, f = 9, th = 1] 3. (3; -) / (1, 3, 0) [g = 6, h = 3, f = 9, th = 8]

2) Thời điểm lấy khoảng cách đầu: xảy ra ở lần lấy thứ 2
- UCS lấy tt (4; a] = (1, 4, 1) (nhánh đi sang phải ăn b trước)
- A_V lấy tt (2; ab] = (1, 2, 3) (nhánh đi sang trái về phía a trước)

* Giải thuật bằng khoảng cách ưu tiên (f, tức hết)
- ở lần lấy 1, từ trạng thái gốc (3; ab) theo thứ tự hướng R D L U, cả 2 thuật đều xét R trước (sinh (4; a) với g = 1, tức hết = 1) rồi mới xét hướng L (sinh ra (2; ab) với g = 1, th = 2)
- Đối với UCS (h = 0 ⇒ g = g) khoảng ưu tiên là (g, th). 2 tt (4; a) và (2; ab) có cùng g = g = 1 nên xử lý bằng th (FIFO) vì (4; a) được đưa trước (th = 1 < th = 2), khoảng (1, 1) < (1, 2) làm UCS lấy (4; a) trước.
- Đối với A_V (f = g + h_v) khoảng ưu tiên (f, th)
+ Tại (4; a) dù đã ăn b, vẫn phải qua trái vs a (1, 1) với a → Cr (1, 6) nên h_v (4; a) = d (4; 1) + d (1, 6) = 3 + 5 = 8 ⇒ f (4; a) = 1 + 8 = 9 khoảng là (9, 1)
+ Tại (2; ab) trên cạnh a (1, 1) nên h_v (2; ab) = d (2, 1) + d (1, 6) = 1 + 5 = 6 ⇒ f (2; ab) = 1 + 6 = 7, khoảng là (7, 2)
+ Vì ưu tiên 1 thì f (2; ab) = 7 < g (4; a) = 9 ⇒ (7, 2) < (9, 1) A_V chọn (2; ab) mà bước 2

* Sinh con R trước có bước A* lấy R trước 0;
- Không, sinh R trước dù giúp R có th (id) nhỏ hơn con L; Trong A* thì là khoảng phụ thuộc giữa đồng hàng hay cùng giá trị f.

3.
- Đến bước sau bước 6 của A_V đến khi kết thúc.
+ Lần 7: Lấy (5; -) = (1, 5, 6) [g = 6, h = 1, f = 7] mở theo hướng R sinh ra tt tức là (6; -) = (1, 6, 0) [g = 7, h = 0, f = 7, chính là (5; -)] (hướng L về (4; -) có g_{mn} = 7 > 5 nên loại

- Lần lấy 8: Lấy (6; -) = (1, 6, 0) [g = 7, h = 0, f = 7] ra khỏi OPEN
kiểm tra lời giải: Đúng tại G(1, 6) và mask = 0 (R = -) thỏa mãn, kết thúc
lại và không mở rộng thêm (popped = 8
expanded = 7)

- Đường đi hoàn chỉnh và chi phí
+ Chuỗi trạng thái
(3; ab) $\xrightarrow{L}$ (2; ab) $\xrightarrow{L}$ (1; b) $\xrightarrow{R}$ (2; b) $\xrightarrow{R}$ (3; b) $\xrightarrow{R}$ (4; -) $\xrightarrow{R}$ (5; -) $\xrightarrow{R}$ (6; -)

(Viết theo bộ ba (r, c, mask)
+ (1,3,3) → (1,2,3) → (1,1,2) → (1,2,2) → (1,3,2) → (1,4,0) → (1,5,0)
→ (1,6,0)

- Dãy hành động: L → L → R → R → R → R → R (Vì a(1,1) ở bước 2, b(1,4) ở bc 5
tới đích G(1,6) ở bước 7)

- Chi phí cost = 7

- Chứng minh tính tối ưu bằng cách đưa được
+ Mọi lộ trình từ p = (1, 3) đều phải đến ô a(1,1) rồi từ a → G
+ Độ lệch Manhattan là cận dưới của số bước di chuyển tối giữa 2 ô trên lưới, tổng
số bc của 1 lộ trình phải ≥ h(s₀):
cost ≥ h(s₀) = d(P, a) + d(a, G) = |3 - 1| + |1 - 6| = 2 + 5 = 7
+ Lộ trình tìm được đúng = cận dưới chặt nhận được h(s₀) = 7
→ Là đường đi tối ưu

- Chỉ ra ô quay lại nhưng trạng thái mới
+ Ô (1,2)
· Lần 1 (bước 1, g = 1) Trạng thái (2; ab) = (1, 2, 3)
· Lần 2 (bước 3, g = 3) Trạng thái (2; b) = (1, 2, 2)
+ Vì lưu ít là cả bộ ba (r, c, mask) chứ không chỉ tọa độ (r, c)
→ Chương trình nhận diện 2 ttt khác nhau nên cho phép thực hiện.

Bài 3:

1,

- Ô cần quay lại: Các ô nằm trên đường đi từ vị trí xuất phát vào ngõ cụt để ăn thức ăn. Sau khi ăn xong, Pacman bắt buộc phải đi ngược lại qua các ô này để về đích G = (1, 6).

- Cách này có thể làm bác không có lời giải vì: mô hình bài toán yêu cầu Pacman phải ăn hết toàn bộ thức ăn rồi mới tới G. Cùng 1 toạ độ (r, c) nhưng tập thức ăn còn lại khác nhau là 2 trạng thái hoàn toàn khác nhau. Nếu thuật toán chỉ dùng visited = {(r, c)} và đánh dấu thăm ngay khi phát hiện toạ độ đơn lẻ, thì khi Pacman đi qua ô (1, 2) lần đầu, ô này bị đánh dấu là đã thăm. Đến khi quay về, đường đi qua ô này sẽ bị chặn đứng dù lúc này tập thức ăn đã khác.

- Khẳng định hiển thị và hụ vọng tính tay:

+ Dữ liệu hiển thị: Bản đồ ngắn dạng hành lang chứa ngõ cụt bắt buộc phải quay đầu.

+ Kỳ vọng tính tay: Nếu cài đặt visited theo toạ độ (r, c), thuật toán sẽ trả về no_path. Ngược lại, với mô hình chuẩn dùng khoá trạng thái dạng đủ (r, c, mask), thuật toán sẽ tìm

ra lộ trình hợp lệ đi sang trái ăn F rồi quay đầu đi qua lại p để đến G với chi phí chính xác.

2,
- Không được dừng lần đầu qua G vì điều kiện hoàn tất yêu cầu đồng thời vị trí ở G và mask = 0. Lần đầu qua G vẫn còn thức ăn phía sau nên chưa được dừng.
- Chi phí tối ưu và số lần ghé G:
+) G là khoảng cách từ p → G là d(p; G) và từ G đến viên thức ăn F là d(G; F)
=) Chi phí tối ưu: Cost = d(p; G) + d(G; F) + d(F; G)
+) Số lần ghé G: Pacman ghé G tổng cộng 2 lần. Lần 1 đi ngang qua trên đường tiến về F khi chưa hết thức ăn, lần 2 quay về và dừng lại hoàn tất ở G khi mask = 0.

3,
- Các giá trị và điều kiện dích:
+) $h_F = 0$ vì $R = \emptyset$
+) $h_V = d(p; G) = 3$
+) Điều kiện dích thực sự: p == G and mask == 0
- Lỗi của if h == 0: return Success: Chỉ kiểm tra h mà không kiểm tra vị trí p = G, dẫn đến việc thuật toán dừng lại ngay tại viên thức ăn cuối cùng, kết quả sai lệch hoàn toàn.

Bài 4:

1, $h_{sum} = d(P;G) + \sum_{F \in R} d(P;F)$:

+ Giá trị ở đầu: $3 + (1+2) = 6$

+ Đánh giá: Chi phí tối ưu thực tế lời cost* = 3. Vì $h_{sum} = 6 > 3$ nên phép cộng này vượt chi phí thực tế, không thoả mãn tính chất chấp nhận được và không phải là cận dưới đảm bảo.

2, - Giá trị khi đã ăn hết, đứng tại G: Vẫn giữ nguyên khoảng cách xa nhất từ G tới tập F_0 ban đầu (khác 0).

- Kiểm tra điều kiện cần: Vi phạm nghiêm trọng điều kiện cần của heuristic chấp nhận được tại đích ($h(goal) = 0$)

3, - no_path: Trạng thái khi hàng đợi biên OPEN đã cạn kiệt mà vẫn không tìm thấy đường đi đến đích, chứng tỏ bài toán thực sự vô nghiệm.

- limit: Trạng thái dừng sớm do thuật toán chạy chạm ngưỡng giới hạn tài nguyên cho phép, chưa thể kết luận chắc chắn có hay không lời giải do bị cắt ngang giữa chừng.

- Ngân sách 0 nhưng đầu đã thoả điều kiện đích: Thuật toán kiểm tra đích ngay khi lấy ra khỏi heap (pop) trước khi xét ngân sách. Do đó vẫn trả về found với cost = 0, popped = 1, expanded = 0.